DATA 604 – Final Project: Comparative Analysis of Representation Methods for Classification

Abstract

This report presents a comparative analysis of different representation methods for image classification using the MNIST dataset of handwritten digits. Four representation methods were evaluated: Raw Data method, Principal Component Analysis (PCA), Kernel Principal Component Analysis (kPCA), and Locally Linear Embedding (LLE). The k-Nearest Neighbors (kNN) algorithm was employed as the classifier across all methods. Performance was assessed using multiple metrics, including accuracy, precision, recall, and F1 score. The experiments revealed that dimensionality reduction techniques can maintain or even improve classification performance while significantly reducing computational demands. Specifically, PCA achieved the highest overall performance (97.23% accuracy) while requiring only a fraction of the computational resources compared to the Raw Data method. This study provides insights into the trade-offs between classification performance, dimensionality reduction, and computational efficiency in the context of handwritten digit recognition.

Introduction

Machine learning applications often face the challenge of high-dimensional data, which can lead to increased computational complexity and the curse of dimensionality. Representation methods, particularly dimensionality reduction techniques, offer a solution by transforming the data into a lower-dimensional space while preserving essential information. This report analyzes four different representation methods for classifying handwritten digits from the MNIST dataset.

The four methods examined are:

- **Raw Data:** Using the original pixel values without dimensionality reduction
- **Principal Component Analysis (PCA):** A linear method that identifies the directions of maximum variance
- **Kernel Principal Component Analysis (kPCA):** A nonlinear extension of PCA that uses the kernel trick
- **Locally Linear Embedding (LLE):** A nonlinear method that preserves local relationships between neighboring points

Each method was evaluated using the k-Nearest Neighbors (kNN) classifier, a nonlinear classification algorithm that assigns class labels based on the majority vote of nearest neighbors in the feature space. The purpose of this analysis is to determine which representation method provides the best balance between classification performance, computational efficiency, and generalization capability.

Dataset Description

The MNIST (Modified National Institute of Standards and Technology) dataset is a widely used benchmark in the field of machine learning, especially for image classification tasks. It consists of 28×28 grayscale images of handwritten digits (0-9). The dataset used in this study includes:

- 60,000 training images (split into 48,000 for training and 12,000 for validation)
- 10,000 test images

Each image has been normalized to fit into a 28×28 pixel bounding box and anti-aliased, resulting in grayscale levels. The images are centered by computing the center of mass of the pixels and translating the image to position this point at the center of the 28×28 field.

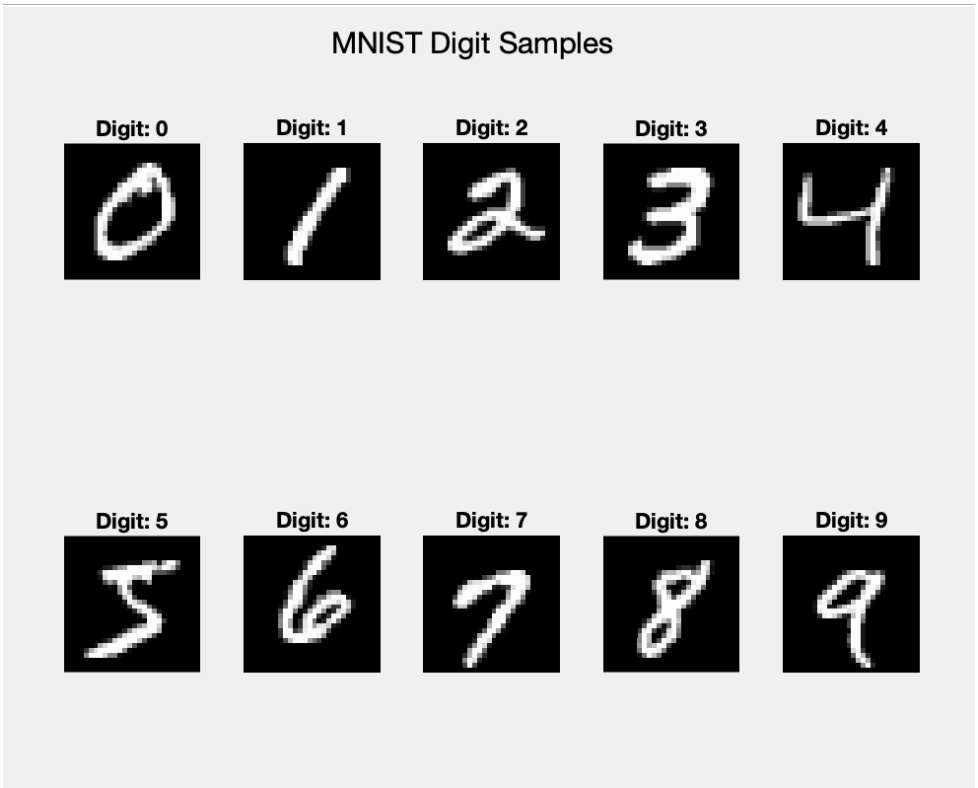

Figure 1: Sample images from the MNIST dataset representing each digit class (0-9)

The dataset presents several challenges typical in real-world pattern recognition tasks:

- Variations in handwriting styles
- Different thicknesses and orientations of the digits
- Varying degrees of clarity and completeness

Each image in the raw format consists of 784 features (28×28 pixels), making this a high-dimensional classification problem.

Methodology

Data Preprocessing

The MNIST dataset was preprocessed as follows:

1. **Data Loading**: The images and labels were loaded from the standard MNIST files (train-images-idx3-ubyte, train-labels-idx1-ubyte, t10k-images-idx3-ubyte, t10k-labels-idx1-ubyte).
2. **Normalization**: The pixel values were normalized to the range [0, 1] by dividing by 255.
3. **Data Splitting**: The original training set (60,000 images) was split into a training set (48,000 images) and a validation set (12,000 images) using a 80-20 split. The test set remained unchanged (10,000 images).
4. **Dimensionality**: Each image was flattened from a 28×28 matrix to a 784-dimensional vector for processing.

Representation Methods

Raw Data

The raw data approach uses the original 784 features (pixel values) without any transformation or reduction. This serves as the baseline for comparison with dimensionality reduction methods.

Principal Component Analysis (PCA)

PCA reduces dimensionality by projecting the data onto a lower-dimensional space defined by the eigenvectors of the data's covariance matrix. The implementation used in this study:

- Retained principal components that explained 80% of the variance
- Resulted in a 44-dimensional representation (reduced from 784 dimensions)

Kernel Principal Component Analysis (kPCA)

kPCA extends PCA to capture nonlinear patterns by implicitly mapping the data to a higher-dimensional space using the kernel trick. The implementation used:

- Radial Basis Function (RBF) kernel with $\sigma = 15$
- Nyström approximation with 20,000 landmarks (2,000 per digit) to handle memory constraints
- Embedding into a 100-dimensional space

Locally Linear Embedding (LLE)

LLE preserves the local structure of the data by representing each point as a linear combination of its neighbors. The implementation used:

- 12 nearest neighbors for reconstruction
- Regularization parameter of 1e-3
- Embedding into a 20-dimensional space
- Used a subset of 30,000 samples (3,000 per digit) due to memory constraints

Classification Approach

The k-Nearest Neighbors (kNN) algorithm was used as the classifier for all representation methods. For each method:

1. **Hyperparameter Tuning**: k values of 1, 3, 5, 7, and 9 were evaluated using 5-fold cross-validation on the training set.
2. **Distance Metric**: Euclidean distance was used for Raw Data, while cosine distance was used for the dimensionality reduction methods.
3. **Final Evaluation**: The best k value from validation was used to train a model on the combined training and validation sets, which was then evaluated on the test set.

Evaluation Metrics

Performance was assessed using multiple metrics:

1. **Accuracy**: The proportion of correctly classified instances
2. **Precision**: The ability of the classifier to avoid false positives, calculated for each class and averaged
3. **Recall**: The ability of the classifier to find all positive instances, calculated for each class and averaged
4. **F1 Score**: The harmonic mean of precision and recall, providing a balance between the two
5. **Confusion Matrix**: Visualizing the classification results across all classes
6. **Computational Efficiency**: Theoretical assessment of computational demands, memory requirements, and algorithmic complexity

Experimental Results

Method 1: Raw Data with kNN

The raw data approach used the original 784 features without any transformation. Cross-validation was performed to determine the optimal number of neighbors (k) for the kNN classifier.

Cross-Validation Results:

- k=1: CV Accuracy=0.9697
- k=3: CV Accuracy=0.9693
- k=5: CV Accuracy=0.9690
- k=7: CV Accuracy=0.9671
- k=9: CV Accuracy=0.9645

Test Results:

- Test Accuracy: 96.91%
- Average Precision: 98.57%
- Average Recall: 96.88%
- Average F1 Score: 97.71%

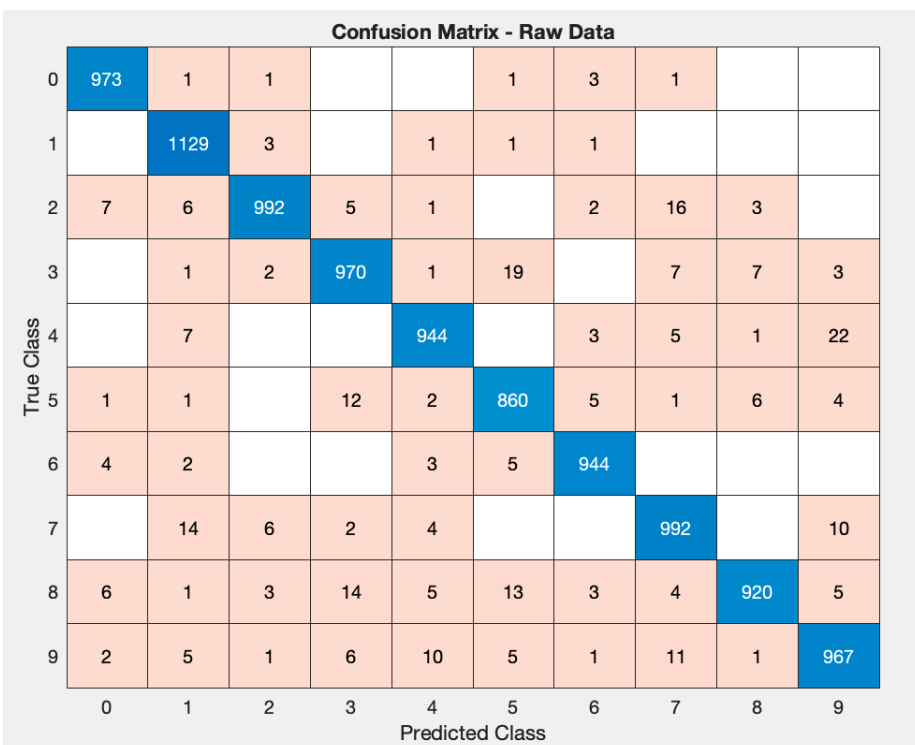

Figure 2: Confusion Matrix for the Raw Data approach

The raw data method achieved strong overall performance but required high computational resources, with a total processing time of 1292.48 seconds (21.54 minutes).

Method 2: PCA with kNN

PCA reduced the dimensionality from 784 to 44 features while preserving 80% of the variance in the data. Cross-validation was performed to determine the optimal k value.

Cross-Validation Results:

- k=1: CV Accuracy=0.9740
- k=3: CV Accuracy=0.9739
- k=5: CV Accuracy=0.9734
- k=7: CV Accuracy=0.9713
- k=9: CV Accuracy=0.9692

Test Results:

- Test Accuracy: 97.23%
- Average Precision: 98.68%
- Average Recall: 97.20%
- Average F1 Score: 97.93%

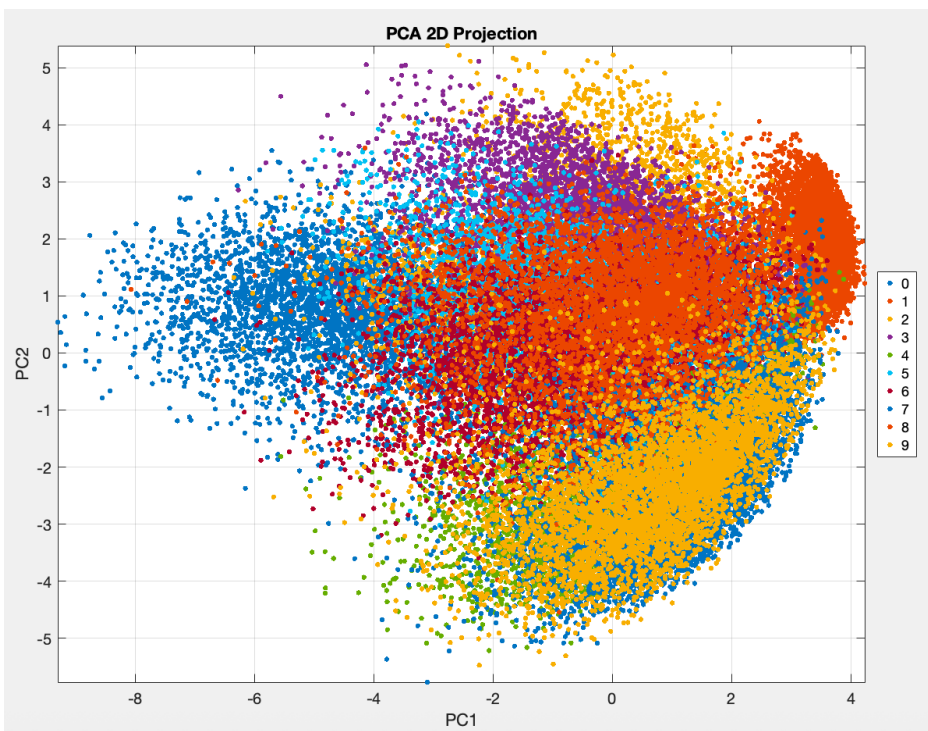

Figure 3: 2D projection of the training data using the first two Principal Components

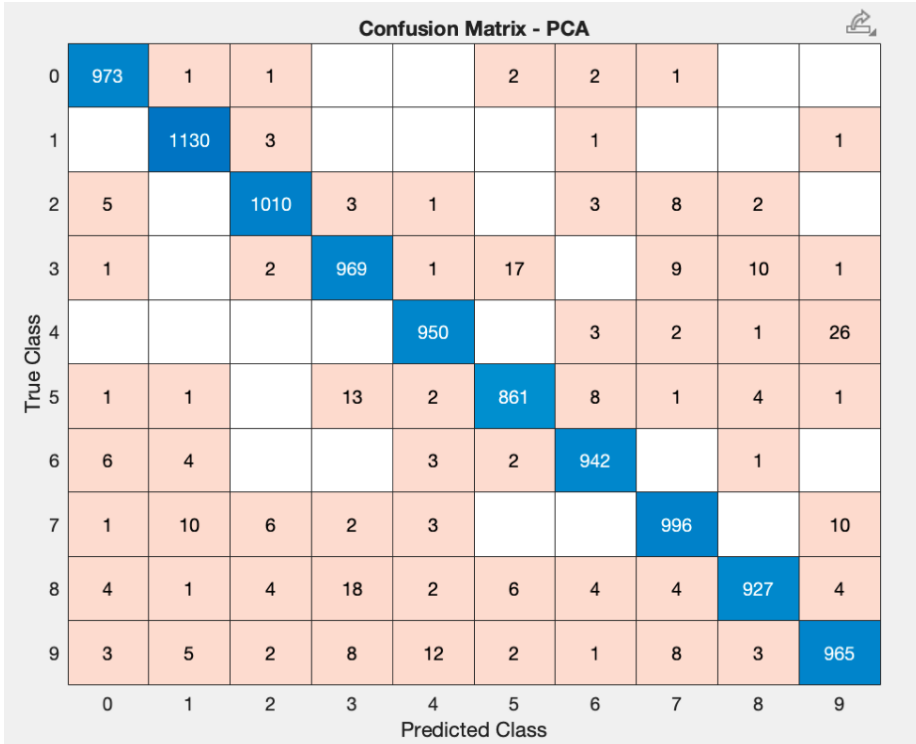

Figure 4: Confusion Matrix for PCA

PCA achieved the highest test accuracy among all methods while significantly reducing computational time to 57.56 seconds (0.96 minutes).

Method 3: Kernel PCA with kNN

Due to memory constraints with the full dataset, Kernel PCA was implemented using the Nyström approximation method with 20,000 landmarks (2,000 per digit). The RBF kernel with σ = 15 was used, and the data was embedded into a 100-dimensional space.

System Limitations: When attempting to run Kernel PCA on a larger subset of the data, the following error was encountered:

```
METHOD 3: KPCA via Nyström WITH kNN
Error using statslib.internal.pdist2mex
Requested 48000x25000 (8.9GB) array exceeds maximum array size preference (8.0GB). This might cause MATLAB to become unresponsive.

Error in statslib.internal.pdist2 (line 443)
            D = statslib.internal.pdist2mex(X',Y',dist,additionalArg,smallestLargestFlag,radius,doSort);

Error in pdist2 (line 158)
[varargout{1:nargout}] = statslib.internal.pdist2(X,Y,varargin{:});

Error in Final2 (line 302)
D2_Ct  = pdist2(X_train, landmarks, 'euclidean').^2;

Related documentation
```

This memory limitation necessitated the use of the Nyström approximation method, which is more memory-efficient for large datasets.

Cross-Validation Results:

- k=1: CV Accuracy=0.9732
- k=3: CV Accuracy=0.9730
- k=5: CV Accuracy=0.9729
- k=7: CV Accuracy=0.9711
- k=9: CV Accuracy=0.9699

Test Results:

- Test Accuracy: 97.12%
- Average Precision: 98.62%
- Average Recall: 97.10%
- Average F1 Score: 97.85%

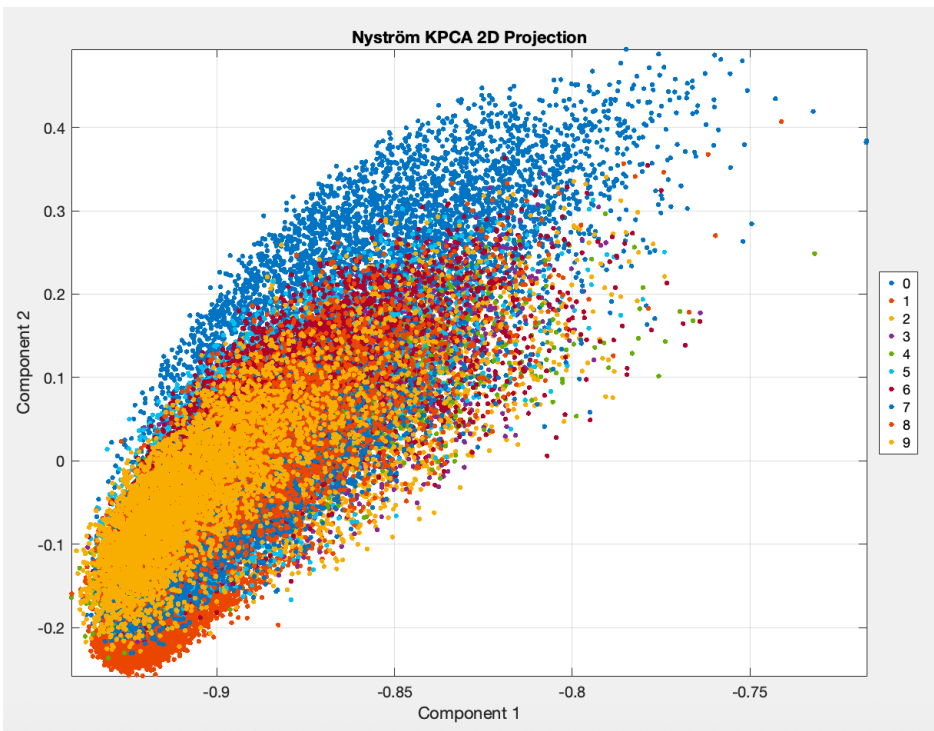

Figure 5: 2D Projection of the training data using the first two Kernel Principal Components

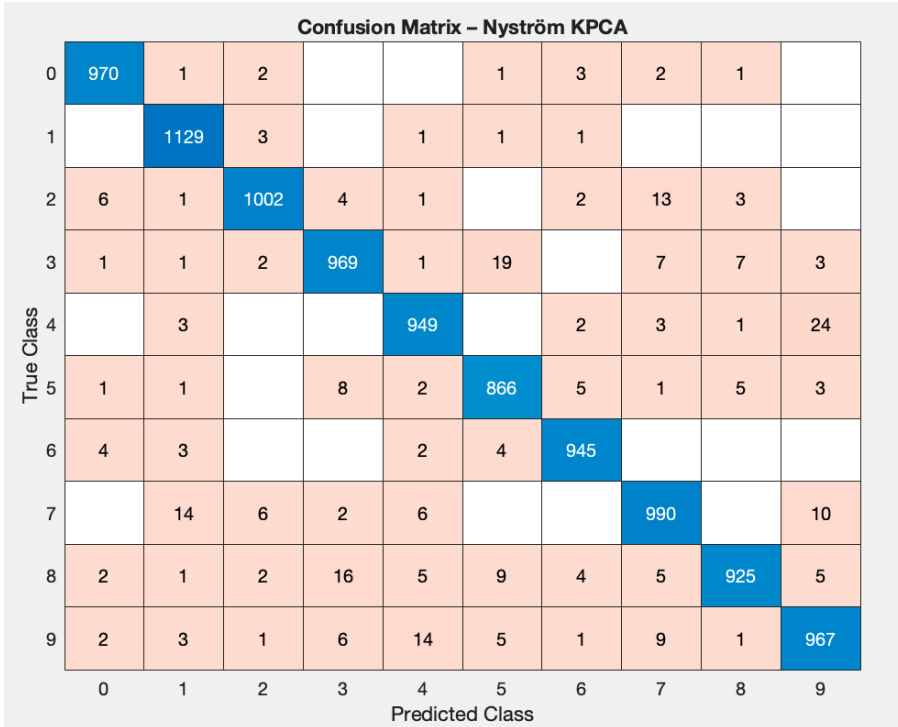

Figure 6: Confusion Matrix for Kernel PCA

Kernel PCA achieved strong performance, slightly below PCA but above Raw Data, with a computational time of 515.77 seconds (8.60 minutes).

Method 4: LLE with kNN

Due to severe memory constraints when applying LLE to the full dataset, a subset of 30,000 samples (3,000 per digit) was used for learning the embedding. The data was embedded into a 20-dimensional space using 12 nearest neighbors for reconstruction.

System Limitations: When attempting to run LLE on a larger subset of the data, the following error was encountered:

```
METHOD 4: LLE with kNN
Finding nearest neighbors...
Compute reconstruction weights...
Compute embedding (solve eigenproblem)...
Error using eye
Requested 33000x33000 (8.1GB) array exceeds maximum array size preference (8.0GB). This might cause MATLAB to become unresponsive.

Error in lle (line 117)
        [mappedX, eigenvals] = eigs(M + eps * eye(n), no_dims + 1, tol, options);          % only need bottom (no_dims + 1) eigenvectors

Error in compute_mapping (line 203)
            else [mappedA, mapping] = lle(A, no_dims, varargin{1}, eig_impl); end

Error in Final2 (line 449)
[~, lle_map] = compute_mapping(X_sub, 'LLE', embed_dim, num_neigh, reg_param);

Related documentation

>>
```

This error occurred because LLE requires storing an n×n matrix in memory, which becomes prohibitively large for datasets with many samples.

Cross-Validation Results:

- k=1: CV Accuracy=0.9486
- k=3: CV Accuracy=0.9544
- k=5: CV Accuracy=0.9549
- k=7: CV Accuracy=0.9544
- k=9: CV Accuracy=0.9537

Test Results:

- Test Accuracy: 95.80%
- Average Precision: 98.27%
- Average Recall: 95.76%
- Average F1 Score: 96.99%

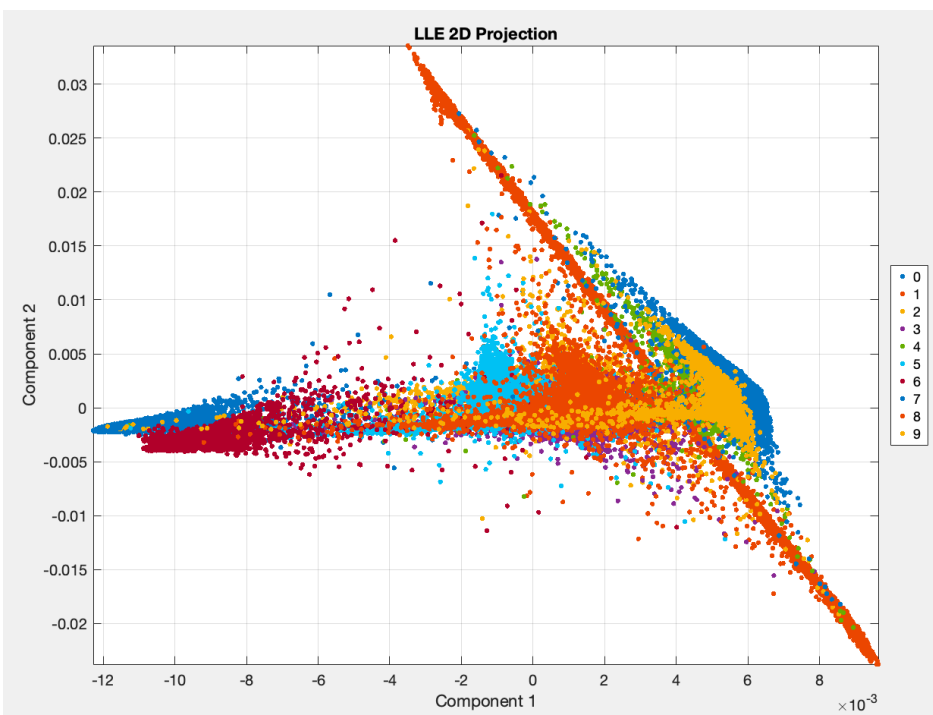

Figure 7: 2D Projection of the training data using the first two LLE Components

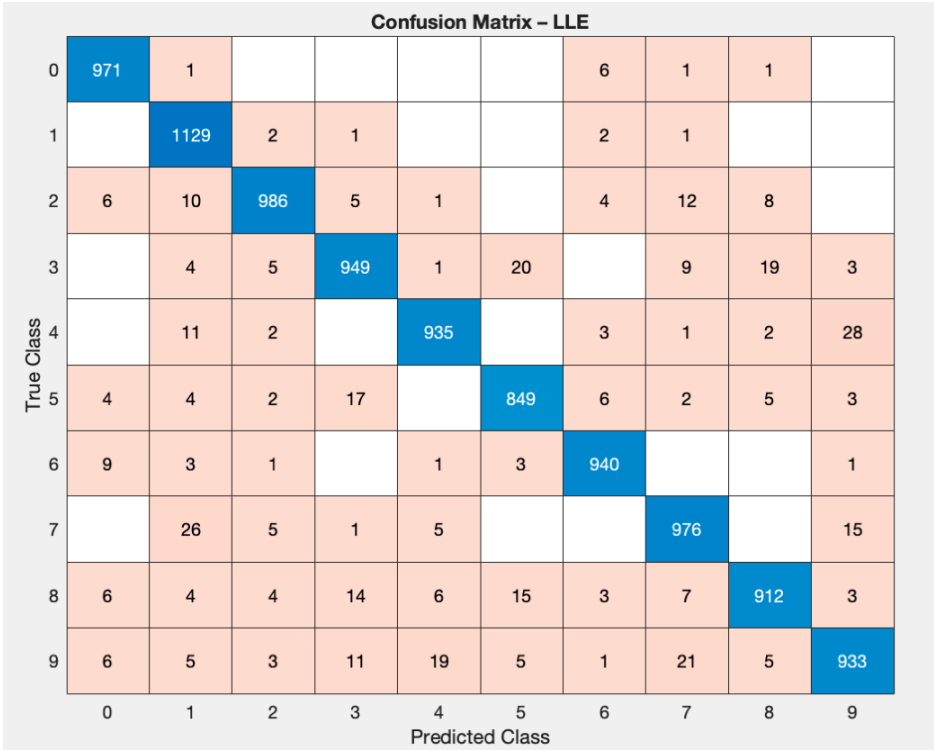

Figure 8: Confusion Matrix for LLE

LLE achieved the lowest accuracy among the four methods, although still above 95%. The computational time was 508.36 seconds (8.47 minutes), similar to Kernel PCA.

Comparative Analysis

Performance Metrics Comparison

Below table presents a comparison of the performance metrics across all four methods:

Table 1: Performance Metrics Comparison

Method	Accuracy	Precision	Recall	F1 Score
Raw Data	0.9691	0.9857	0.9688	0.9771
PCA	0.9723	0.9868	0.9720	0.9793
Kernel PCA	0.9712	0.9862	0.9710	0.9785
LLE	0.9580	0.9827	0.9576	0.9699

PCA consistently outperformed the other methods across all metrics, achieving the highest accuracy (97.23%), precision (98.68%), recall (97.20%), and F1 score (97.93%). Kernel PCA was a close second, followed by Raw Data and then LLE.

It's noteworthy that PCA, despite reducing the dimensionality from 784 to just 44 features (a 94.4% reduction), actually improved classification performance compared to using the raw data. This suggests that PCA effectively captured the most discriminative information while filtering out noise.

Computational Efficiency

The four representation methods show significant differences in computational efficiency, which is an important consideration for practical applications.

Raw Data requires processing the full 784-dimensional feature vectors during classification. This leads to:

- Increased memory usage for storing high-dimensional distance matrices
- Computational intensity when calculating distances between points in high-dimensional space
- The curse of dimensionality affecting the kNN algorithm, as distance metrics become less meaningful in very high dimensions

PCA offers substantial computational advantages:

- Reduces dimensionality from 784 to just 44 features (94.4% reduction)
- Requires an initial eigendecomposition of the covariance matrix, but this is a one-time cost
- Subsequent operations work in a much lower-dimensional space, greatly accelerating the kNN classification
- Improves memory efficiency due to the compact data representation
- Matrix operations for projection can be highly optimized using linear algebra libraries

Kernel PCA with Nyström Approximation presents mixed computational characteristics:

- Avoids explicit computation of the full kernel matrix (which would be prohibitively large for 60,000 samples)
- Still requires significant memory for the landmark-based approximation
- Computing distances between points in kernel space adds overhead
- The memory constraints necessitated the use of 20,000 landmarks rather than the full dataset

LLE shows the most significant computational limitations:

- Requires finding nearest neighbors for each data point
- Involves solving a large eigenvalue problem
- Memory requirements scale poorly with dataset size, limiting applicability to large datasets
- The memory constraints required reducing the dataset size to 30,000 samples

From a computational complexity perspective, PCA offers the best balance between performance and efficiency for this dataset. Raw data processing becomes increasingly inefficient as dimensionality grows, while both Kernel PCA and LLE face practical scalability limitations for large datasets due to their memory requirements and algorithmic complexity.

These computational considerations are important when deploying machine learning systems in resource-constrained environments or when processing very large datasets, highlighting the practical advantages of efficient dimensionality reduction techniques like PCA.

Visualization and Separability

The 2D projections provide insights into how each method separates the digit classes:

1. **PCA**: The 2D projection (Figure 3) shows clear clustering of digits, with some overlap between similar digits like 4/9 and 3/5. The feature space is efficiently used with good spread across both dimensions.
2. **Kernel PCA**: The 2D projection (Figure 5) shows improved separation for some digit pairs compared to PCA, particularly for digits 0 and 1, which form more distinct clusters. This suggests that the nonlinear mapping captures relationships that linear PCA cannot.
3. **LLE**: The 2D projection (Figure 7) shows a more complex structure with some digits forming elongated clusters. While there is separation between clusters, the boundaries appear less defined than in PCA and Kernel PCA.

These visualizations demonstrate that while 2D projections are insufficient for optimal classification (hence the use of higher-dimensional embeddings for the actual classification), they provide valuable insights into how each method transforms the data space.

Class-wise Performance

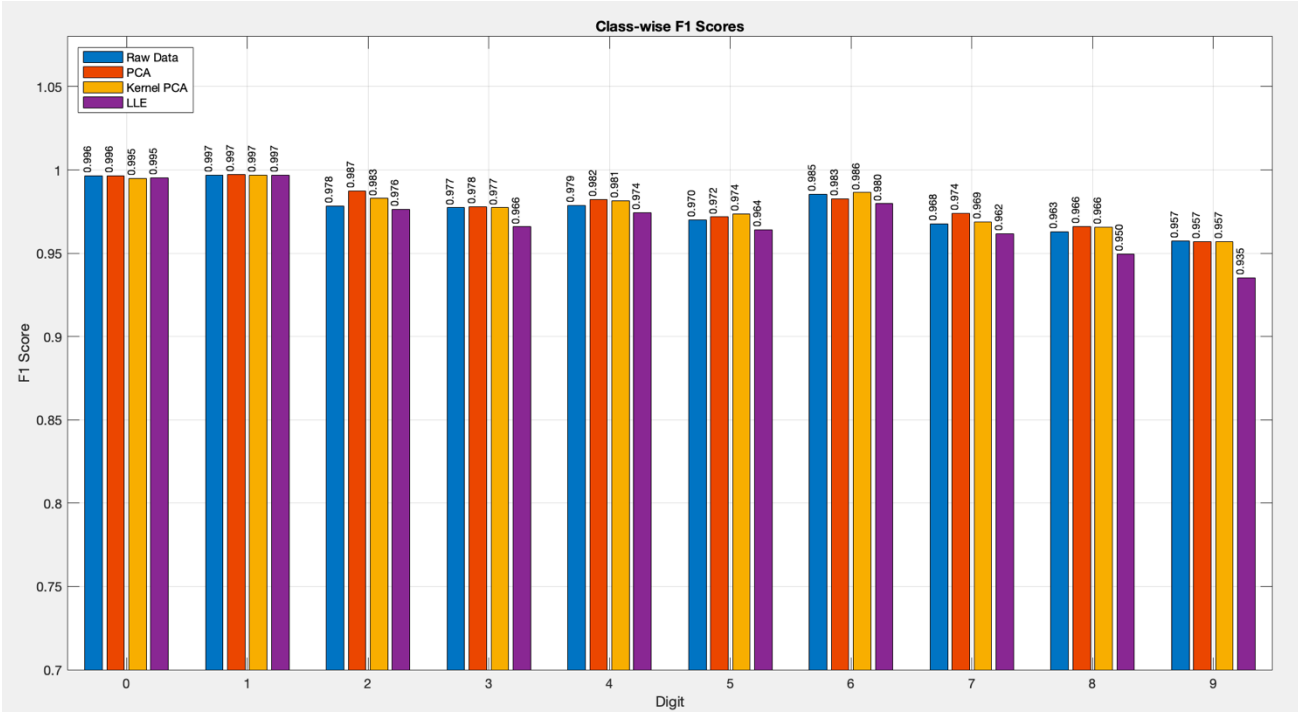

Figure 9: Class-wise F1 score for all four methods

Several patterns emerge from the class-wise analysis:

1. All methods performed best on digit '1', which is the most distinctive digit with the simplest structure.
2. Digits '0', '1', and '6' were consistently classified well across all methods, with F1 scores above 98%.
3. The most challenging digits were '8' and '9', which showed the lowest F1 scores across all methods. This is likely due to their similarity to other digits (e.g., '8' can resemble '3' or '5', and '9' can resemble '4' or '7').
4. LLE performed notably worse than the other methods on digit '9', with an F1 score approximately 2% lower than the other approaches.
5. PCA and Kernel PCA showed similar class-wise performance patterns, suggesting that for this dataset, the nonlinear mapping provided by Kernel PCA offered limited additional benefit over linear PCA.

Conclusion

This study evaluated four representation methods—Raw Data, PCA, Kernel PCA, and LLE—for classifying handwritten digits using the MNIST dataset. Several key findings emerged:

1. **PCA achieved the best overall performance**: With the highest accuracy (97.23%) and F1 score (97.93%), PCA demonstrated that effective dimensionality reduction can actually improve classification performance by filtering out noise while preserving discriminative information.
2. **Computational efficiency varies dramatically**: PCA offers substantial computational advantages over the other methods by reducing the dimensionality to just 44 features (94.4% reduction). Raw data processing, by contrast, must handle the full 784-dimensional space, resulting in lot higher computational demands during classification.
3. **Nonlinear methods faced practical limitations**: Both Kernel PCA and LLE encountered memory constraints with the full dataset, requiring approximations (Nyström method) or subset selection. This shows a practical limitation of these methods for large datasets.
4. **The simplest method is often competitive**: Raw Data method, while computationally expensive, achieved strong performance (96.91% accuracy), demonstrating that in some cases, sophisticated dimensionality reduction may not be necessary.
5. **Different digits benefit from different representations**: While all methods performed well on distinctive digits like '0' and '1', the performance varied more for complex digits like '8' and '9', suggesting that hybrid approaches might be beneficial for handling diverse patterns.

Based on these findings, we can make the following recommendations:

1. **PCA as the first choice**: For handwritten digit classification, PCA offers the best balance between performance, computational efficiency, and simplicity. It should be the first method considered for similar image classification tasks.
2. **Consider the practical constraints**: When working with large datasets, memory and computational constraints may limit the applicability of nonlinear methods like Kernel PCA and LLE. PCA offers a more scalable alternative.
3. **Evaluate multiple metrics**: While accuracy is commonly reported, a comprehensive evaluation should include precision, recall, and F1 score, particularly for imbalanced datasets or when the cost of different types of errors varies.
4. **Class-specific analysis**: Examining performance by class reveals patterns that may be obscured by aggregate metrics and can guide the selection of appropriate methods for specific recognition tasks.

References

1. DATA 604 Course Materials, Spring 2025.
2. MNIST Dataset. Retrieved from https://www.kaggle.com/datasets/hojjatk/mnist-dataset
3. Roweis, S. T., & Saul, L. K. (2000). Nonlinear dimensionality reduction by locally linear embedding. Science, 290(5500), 2323-2326.
4. Williams, C. K., & Seeger, M. (2001). Using the Nyström method to speed up kernel machines. Advances in neural information processing systems, 14.

The implementation code can be found in the submission portal's comments section.